CULTURE & CONTEXT — COMPLETE PROJECT SPECIFICATION

DOCUMENT PURPOSE

This is the comprehensive technical and product specification for **Culture & Context**, an AI-powered creative direction SaaS tool for the music industry. This document contains everything needed to continue development: architecture, file structure, all decisions made, current state, what works, what's broken, and what's next.

---

1. PRODUCT OVERVIEW

What It Is

Culture & Context is a creative direction engine. A user (music industry creative director, marketing team, artist manager, label A&R) inputs reference material — mood board images, Pinterest boards, YouTube videos, artist photos, and text descriptions — and the system:

- **Analyzes** all inputs using Claude Vision (image analysis) and Claude Sonnet (text/synthesis)
- **Synthesizes** a unified creative direction from the analysis
- **Generates AI images** placing the artist into the visual world using Gemini (photography, video stills, cover art, merch mockups, logo concepts, stage design)
- **Produces a polished ~45-slide deck** (PDF + PPTX) with creative direction, touchpoint recommendations, and visual concepts
- **Cross-references a cultural graph database** (Neo4j) to find talent recommendations (photographers, directors, stylists) and cultural connections

The Name

- **Culture & Context** — the product/company name
- **Graft** — the graph visualization portal (a feature within the product, not the product itself)

Target Users

- Creative directors at record labels
- Artist managers building visual identities
- Marketing teams at labels/agencies
- Independent artists wanting professional creative direction
- Brand partnership teams matching artists with brands

Business Model (Future)

- Per-campaign pricing (~$15-25 per campaign, costs ~$3-5 to run)
- Monthly subscription for agencies/labels

- Enterprise tier with custom graph data and integrations

---

2. ARCHITECTURE

Tech Stack

- **Monorepo**: Turborepo
- **Frontend**: Next.js 14 (App Router), React, TypeScript
- **Backend API**: Express.js, TypeScript
- **Database**: Neo4j (graph database) via Homebrew service
- **AI - Analysis**: Anthropic Claude (claude-sonnet-4-5-20250929) via `@anthropic-ai/sdk`
- **AI - Image Generation**: Google Gemini (gemini-2.5-flash-image) via `@google/genai`
- **PDF Generation**: Puppeteer (HTML→PNG) + pdf-lib (merge into PDF)
- **PPTX Generation**: pptxgenjs (PDF pages→PPTX slides)
- **Pinterest Scraping**: pinterest-dl (Python CLI, subprocess)
- **Data Enrichment**: Wikidata, MusicBrainz, Discogs (free APIs), plus optional TMDb, Last.fm, Spotify

Project Root

```
/Users/moshii/.openclaw/workspace/cultural-graph/
```

Directory Structure

```
cultural-graph/
├── apps/
│   ├── api/                          # Express backend
│   │   ├── src/
│   │   │   └── index.ts              # All API routes (campaigns, graph, ingestion)
│   │   ├── package.json
│   │   └── tsconfig.json
│   └── web/                          # Next.js frontend
│       ├── src/app/
│       │   ├── layout.tsx            # Root layout with nav bar
│       │   ├── page.tsx              # Landing page (Culture & Context hero)
│       │   ├── globals.css           # Global styles
│       │   ├── create/
│       │   │   └── page.tsx          # Campaign creation (2-step: input → curate)
│       │   ├── campaign/
│       │   │   └── [id]/
│       │   │       └── page.tsx      # Campaign results (tabs: Overview, Deck, Images, Graph)
│       │   └── graph/
│       │       └── page.tsx          # Graft — 3D graph visualization
│       ├── next.config.js
│       ├── package.json
│       └── tsconfig.json
├── packages/
│   ├── analysis-engine/              # All AI analysis logic
│   │   ├── src/
│   │   │   ├── index.ts             # Exports
│   │   │   ├── types.ts             # TypeScript types (ImageAnalysis, MoodBoardAnalysis, etc.)
│   │   │   ├── image-analyzer.ts    # Single image → Claude Vision → ImageAnalysis
│   │   │   ├── batch-analyzer.ts    # Up to 200 images → MoodBoardAnalysis + synthesis
│   │   │   ├── youtube-analyzer.ts  # YouTube URL → transcript + visual analysis
```

```
│   │   │   ├── pinterest-scraper.ts # Pinterest board URL → downloaded images
│   │   │   ├── image-generator.ts   # Gemini image generation (25 scenes across 6 categories)
│   │   │   ├── entity-ingestion.ts  # Claude researches entity → creates Neo4j nodes + relationships
│   │   │   └── data-enrichment.ts   # Cross-reference Wikidata, MusicBrainz, Discogs, TMDb, Last.fm, S
│   │   ├── package.json
│   │   └── tsconfig.json
│   ├── deck-generator/               # Deck/PDF/PPTX generation
│   │   ├── src/
│   │   │   ├── index.ts
│   │   │   ├── templates.ts         # 29 slide templates as programmatic HTML/CSS functions
│   │   │   └── assembler.ts         # Orchestrates: templates → Puppeteer → PNG → pdf-lib → PDF
│   │   ├── package.json
│   │   └── tsconfig.json
│   ├── graph-db/                     # Neo4j connection + queries
│   │   ├── src/
│   │   │   ├── index.ts
│   │   │   ├── connection.ts        # Neo4j driver singleton
│   │   │   ├── schema.ts            # Constraints + indexes (13 node types)
│   │   │   └── queries.ts           # CRUD, traversal, cultural queries, talent matching
│   │   ├── package.json
│   │   └── tsconfig.json
│   └── shared/                       # Shared types
│       ├── src/
│       │   ├── index.ts
│       │   └── types/
│       │       └── graph.ts
│       ├── package.json
│       └── tsconfig.json
├── data/
│   ├── seed/
│   │   ├── nodes.json               # 25 original seed profiles
│   │   ├── relationships.json       # 63 seed relationships
│   │   └── mass-ingest.txt          # 114 entities for batch ingestion
│   └── campaigns/                    # Persisted campaign data
│       └── [campaign-id]/
│           ├── campaign.json         # Full campaign state
│           └── deck.pdf              # Generated PDF
├── scripts/
│   ├── import-seed.ts               # Import seed data into Neo4j
│   └── ingest.ts                    # CLI: npx tsx scripts/ingest.ts "Artist Name"
├── .env                              # API keys (ANTHROPIC_API_KEY, GEMINI_API_KEY)
├── .env.example                      # Template
├── .gitignore
├── package.json                      # Turborepo root
├── turbo.json
├── tsconfig.base.json
└── README.md
```

---

3. NEO4J GRAPH DATABASE

Connection

- **URI**: bolt://localhost:7687
- **User**: neo4j
- **Password**: culturalgraph
- **Browser**: http://localhost:7474
- **Runs as**: Homebrew service (`brew services start neo4j`)

Node Types (13 labels)

Each has a unique constraint on `id` field:

LabelDescriptionCount (pre-mass-ingest) |-------|----------|----------| ArtistMusicians, bands, performers37 PhotographerPhotography professionals9 DirectorFilm/video directors8 StylistFashion stylists3 DesignerFashion/graphic designers1 BrandFashion, luxury, streetwear brands19 CityGeographic cultural centers7 SceneVenues, festivals, movements, subcultures10 AestheticVisual/cultural aesthetics12 GenreMusic genres6 ProjectSpecific works/albums/films0 TechniquePhotography/video techniques4 InfluencerSocial media influencers, content creators1 (Emma Chamberlain)

Total pre-mass-ingest: ~116 nodes. A batch ingestion of 114 more entities was started but may not have completed.

Fulltext Index

```
CREATE FULLTEXT INDEX node_name_search IF NOT EXISTS
FOR (n:Artist|Photographer|Director|Stylist|Designer|Brand|City|Scene|Aesthetic|Genre|Project|Technique
ON EACH [n.name]
```

Relationship Types

```
COLLABORATED_WITH, SHOT_BY, DIRECTED_BY, STYLED_BY,
BRAND_AFFILIATION, AESTHETIC_AFFINITY, GEOGRAPHIC_ANCHOR,
PART_OF_SCENE, GENRE_AFFINITY, USES_TECHNIQUE,
COLOR_SIMILARITY, SONIC_PROXIMITY, CULTURAL_BRIDGE, SIMILAR_TO
```

Node Properties (typical for Artist)

```
{
  id: string;                    // kebab-case unique ID
  name: string;                  // Display name
  description: string;           // 1-3 sentence summary
  extended_description: string;  // Full cultural context (100-200 words)
  color_palette: string[];       // 5 hex colors
  color_weights: number[];       // Weight per color
  color_temperature: number;     // -1.0 to 1.0
  color_saturation: number;      // 0-1
  color_value: number;           // 0-1
  aesthetic_tags: string[];      // e.g. ["brutalist", "minimalist", "neon"]
  aesthetic_weights: number[];
  framing: string[];             // e.g. ["close-up", "wide-angle"]
  lighting: string[];            // e.g. ["harsh-flash", "golden-hour"]
  texture: string[];
  medium: string[];
  density: number;               // 0-1
  negative_space: number;        // 0-1
  realism: number;               // 0-1
  post_processing: number;       // 0-1
  era_references: string[];
  mood_tags: string[];
  mood_weights: number[];
  energy: number;                // 0-1
  attitude: string[];
  motifs: string[];
  character_archetype: string;
  world_setting: string;
  // Artist-specific:
  genres: string[];
  bpm_range: number[];
  sonic_energy: number;
```

```
  sonic_valence: number;
  sonic_tags: string[];
  fashion_brands: string[];
  fashion_vocabulary: string[];
  fashion_price_tier: string;
  origin_city: string;
  current_city: string;
  gaze_pattern: string;
  body_language: string;
  // Metadata:
  source: string;                // "automated" | "automated-stub" | "enrichment-stub" | "seed"
  confidence: number;            // 0-1
  created_at: string;            // ISO timestamp
  updated_at: string;
  // Enrichment data (if enriched):
  enrichment_sources: string;    // "wikidata,musicbrainz,discogs"
  enriched_at: string;
  wikidata_id: string;
  musicbrainz_id: string;
  discogs_id: string;
  tmdb_id: string;
  lastfm_tags: string;
}
```

---

4. CAMPAIGN FLOW (THE CORE PRODUCT)

Step 1: Input (Create Page — `/create`)

User provides any combination of:

- **Pinterest board URL** → scraped via pinterest-dl, all images downloaded
- **Image uploads** (drag & drop or file picker) → up to 200 mood board images
- **Artist photos** → up to 10 photos of the artist (separate field, used for AI image generation)
- **Artist name** → text field
- **YouTube URLs** → analyzed for sonic/visual content
- **Text description** → freeform creative brief

The Pinterest scraper downloads all images from a board. There's also an iframe preview ("Preview Board" button) that shows the Pinterest board inline.

Step 2: Curate (Same Page, Step 2)

After uploading/scraping, user sees a **grid of all images**. They can:

- **Star 5-10 favorites** — these become hero references in the deck
- **Deselect images** they don't want analyzed
- All images are still analyzed by Claude Vision, but starred ones get prominent deck placement

Step 3: Analysis (Backend)

- **Image Analysis**: Each image → Claude Vision → structured `ImageAnalysis` (colors, mood, composition, aesthetic tags, etc.). Batch processes up to 200 images. Max tokens 16384 for synthesis.
- **YouTube Analysis**: Extract transcript + screenshot frames → Claude analyzes sonic and visual elements
- **Synthesis**: All individual analyses → Claude synthesizes into unified `MoodBoardAnalysis`:

- Core color palette with weights - Dominant aesthetic tags - Mood/energy profile - Visual composition patterns - 25+ touchpoint recommendations across categories: - photography, music_video, short_form_video, long_form_video,

album_cover, single_cover, social_content, editorial, tour_visuals, merchandise, packaging, typography, logo_design, motion_graphics, stage_design, lighting_design, wardrobe, set_design, color_grading, aspect_ratios, print_campaign, digital_campaign, brand_partnerships, experiential, documentary, bts_content - Manifesto (2-3 sentence creative direction statement) - Extended narrative (full creative direction paragraph)

Step 4: AI Image Generation (Backend)

Using Gemini `gemini-2.5-flash-image`:

- **Formula**: Mood board images (style references) + artist photos (identity references) + text prompt → Gemini generates composite images
- **25 images across 6 sections**:

- Photography (6): editorial portrait, fashion editorial, environmental portrait, intimate close-up, movement/action, conceptual art - Video (6): music video hero, performance, narrative, visual effects, behind-the-scenes, short-form vertical - Cover Art (3): album cover, single cover, deluxe/special edition - Merch (4): t-shirt, hoodie, poster, accessories - Logo (4): primary wordmark, icon/monogram, tour/event, social avatar - Live Performance (2): stage design, festival moment

- Style refs rotated across scenes for variety (4-6 mood board images per scene)
- Artist photos included as identity refs (2-3 per scene)
- Cost: ~$0.02 per image

Step 5: Deck Generation (Backend)

- **29 TypeScript template functions** in `templates.ts` generate HTML/CSS for each slide type
- Templates include: titleSlide, manifestoSlide, colorPalette, moodGrid, typographySlide, photographyDirection, videoDirection, bulletList, packagingMockup, tourPromo, videoFormats, etc.
- **Assembler** (`assembler.ts`):

1. Generates HTML for ~45 slides across 13 sections 2. Uses `getGenImagesBySection()` helper to map AI-generated images to their corresponding deck sections 3. Puppeteer renders each slide to PNG (1920×1080) 4. pdf-lib merges all PNGs into a single PDF

- **PPTX export**: Separate endpoint converts PDF pages to PNGs and assembles into PPTX using pptxgenjs

Step 6: Results (Campaign Page — `/campaign/[id]`)

- **Sticky header** with campaign name + Download PDF + Download PPTX buttons
- **Tabs**:

- **Overview**: Manifesto, color palette, aesthetic tags, mood profile, touchpoint recommendations (2-column grid), talent recommendations from Neo4j - **Deck Preview**: Embedded PDF viewer or slide-by-slide preview - **Generated Images**: Grid of all 25 AI-generated images organized by section - **Graph**: Campaign-specific graph visualization showing matched cultural entities

Persistence

- Campaigns persist to disk: `data/campaigns/[campaign-id]/campaign.json` + `deck.pdf`
- Loaded on server startup: "Loaded N campaigns from disk"
- Survives server restarts

---

5. API ENDPOINTS

All on Express server, port 4000.

Campaigns

```
POST   /api/campaigns                   # Create new campaign (multipart: images, artist_photos, pinte
GET    /api/campaigns                   # List all campaigns
GET    /api/campaigns/:id              # Get campaign details
POST   /api/campaigns/:id/analyze      # Trigger analysis
POST   /api/campaigns/:id/generate     # Trigger AI image generation
POST   /api/campaigns/:id/deck         # Generate deck PDF
GET    /api/campaigns/:id/deck.pdf     # Download PDF
GET    /api/campaigns/:id/deck.pptx    # Download PPTX (converts PDF→PPTX on the fly)
```

Graph

```
GET    /api/graph/visualization         # All nodes + edges for Graft visualization
GET    /api/graph/search?q=query        # Fulltext search nodes
GET    /api/graph/node/:id              # Single node details
GET    /api/graph/node/:id/connections  # Node's connections
```

Ingestion

```
POST   /api/ingest                      # Ingest a new entity { name, label }
```

---

6. ENTITY INGESTION PIPELINE

How It Works

- **Claude researches** the entity: Given a name + optional type, Claude generates a comprehensive cultural profile (description, colors, aesthetics, mood, connections to other entities)
- **Node created** in Neo4j with full properties
- **Connected entities**: Claude identifies 10-20 real connections. Each gets a stub node created (if it doesn't exist) and a typed relationship
- **Data enrichment** (NEW): After Claude's profile, the system cross-references external sources:

- **Wikidata** (free, no key): SPARQL queries for collaborators, record labels, genres, birth place - **MusicBrainz** (free, no key): Artist relationships, label history, production credits - **Discogs** (free, no key): Album credits, visual collaborators, member lists - **TMDb** (free key): Film/video credits, director connections - **Last.fm** (free key): Similar artists, genre tags - **Spotify** (free key): Related artists, genre classification

- **Enrichment connections**: Any connections found by external sources that Claude missed get added as `SIMILAR_TO` relationships
- **Recursive ingestion**: At depth > 1, connected entities also get fully researched (limited to 5 per level to avoid explosion)

CLI Usage

```
SINGLE ENTITY
```

```
npx tsx scripts/ingest.ts "Billie Eilish"

# WITH EXPLICIT TYPE

npx tsx scripts/ingest.ts "Nick Knight" --label Photographer

# BATCH FROM FILE

npx tsx scripts/ingest.ts --file data/seed/mass-ingest.txt
```

Enrichment API Keys (in .env)

```
# REQUIRED

ANTHROPIC_API_KEY=sk-ant-...        # For Claude (analysis + ingestion)
GEMINI_API_KEY=AIzaSy...             # For Gemini image generation

# OPTIONAL (ENRICHMENT — FREE TO OBTAIN)

TMDB_API_KEY=                        # themoviedb.org/signup
LASTFM_API_KEY=                      # last.fm/api/account/create
DISCOGS_TOKEN=                       # discogs.com/settings/developers
SPOTIFY_CLIENT_ID=                   # developer.spotify.com
SPOTIFY_CLIENT_SECRET=

# NO KEY NEEDED:

# WIKIDATA, MUSICBRAINZ — FULLY FREE PUB
```

---

7. FRONTEND DETAILS

Fonts

- **Unbounded** (--font-unbounded): Display/heading font. Bold, wide, 60s mod feel. Used for nav, headings, buttons.
- **Inter** (--font-inter): Body text
- **JetBrains Mono** (--font-jetbrains): Monospace/code elements

Design Direction

- **Dark theme**: Pure black background (#000), white text
- **Monochromatic**: Especially the Graft — all white on black, no colors
- **Typography-forward**: Big, bold, uppercase headings
- **Brutalist + emotional**: Not cold corporate. Andrew's brief: "inspiring, creative, drag and drop, feel something, photo rich, fashion, brutalist because oppression"
- **Spectrum gradient**: Landing page has a slowly rotating rainbow gradient overlay on the image grid

Navigation

Persistent nav bar (sticky top):

- Left: "Culture & Context" wordmark (links to /)
- Right: "+ New" (→ /create) | "History" (→ /) | "Graft" (→ /graph)

Landing Page (`/`)

- Full-viewport hero section
- Living image grid background (24 Unsplash editorial/music/fashion photos)
- Images reveal with staggered animation, randomly swap for movement
- Rotating spectrum gradient overlay + dark vignette
- "CULTURE & CONTEXT" title centered
- No campaigns list (moved to History page — needs to be built as separate route)

Create Page (`/create`)

- 2-step flow
- Step 1: Input fields (Pinterest URL with iframe preview, image uploads, artist photos, artist name, YouTube URLs, description)
- Step 2: Image curation grid (star favorites, deselect unwanted)
- Submit triggers analysis → generation → deck creation

Campaign Page (`/campaign/[id]`)

- Sticky header with campaign name + download buttons (PDF + PPTX/Keynote)
- 4 tabs: Overview, Deck Preview, Generated Images, Graph
- Overview includes touchpoints as 2-column inline grid

Graft Page (`/graph`)

- **3D perspective visualization** — camera floating in space looking at a sphere of cultural entities
- All nodes placed on a sphere surface using Fibonacci distribution
- **True perspective projection**: closer = bigger, farther = smaller (like real life)
- **Monochromatic**: All white on black, no colors

- **Lines**: Bold on sphere edges, fade toward center. Width scales with perspective.
- **Progressive disclosure**: Tier 0 (top 5% by connections) visible first, lower tiers appear as you zoom
- **Interaction**: Scroll to zoom (camera moves closer/farther), drag to rotate, click node → flies to center with connections fanned out, 2nd-degree connections faintly visible
- **Reference point input**: Centered search bar at top — type to find an entity, select it as your anchor
- **Detail panel**: Right side panel showing entity name, type, connections count, description
- **Controls**: +/- keys, double-click to reset, ESC to clear, 0 to reset zoom
- Built with HTML Canvas (2D context), no WebGL/Three.js

---

8. KEY FILES — WHAT EACH ONE DOES

`packages/analysis-engine/src/image-analyzer.ts`

Takes a single image (base64 or URL) → sends to Claude Vision → returns structured `ImageAnalysis` with color palette, mood, composition, aesthetic tags, lighting, texture, etc.

`packages/analysis-engine/src/batch-analyzer.ts`

Takes up to 200 images → analyzes each individually → synthesizes all into a unified `MoodBoardAnalysis`. The synthesis prompt asks Claude to find patterns across all images and produce a cohesive creative direction. Max tokens bumped to 16384 to avoid truncation. Has JSON repair fallback for truncated responses.

`packages/analysis-engine/src/image-generator.ts`

Gemini image generation. 25 scene categories organized by section. Each scene has:

- A detailed text prompt incorporating the synthesis data (colors, mood, aesthetic)
- Style reference images (4-6 mood board images, rotated per scene)
- Artist identity reference images (2-3 artist photos)

Formula: `mood board refs + artist photos + tailored prompt → Gemini generates composite`

`packages/analysis-engine/src/entity-ingestion.ts`

The auto-ingestion engine. Give it a name, Claude researches a full cultural profile, creates the node + 10-20 connections in Neo4j. Now includes data enrichment step that cross-references Wikidata, MusicBrainz, Discogs, etc.

`packages/analysis-engine/src/data-enrichment.ts`

NEW — Cross-references external data sources. Each function is independent:

- `searchWikidata(name)` → entity ID, description, structured claims
- `getWikidataRelationships(entityId)` → SPARQL query for collaborators, labels, genres
- `searchMusicBrainz(name)` → artist relationships, label history
- `searchDiscogs(name)` → album credits, members
- `searchTMDb(name)` → film/video credits (needs API key)
- `getLastFmSimilar(name)` → similar artists (needs API key)
- `getSpotifyRelated(name)` → related artists (needs API key)
- `enrichEntity(name, label)` → runs all in parallel, returns unified result

`packages/deck-generator/src/templates.ts`

29 slide templates as TypeScript functions that return HTML strings. Each template takes synthesis data and returns a styled slide. Templates use inline CSS for portability.

`packages/deck-generator/src/assembler.ts`

Orchestrates deck generation:

- Takes synthesis data + generated images
- Calls template functions to produce HTML for ~45 slides across 13 sections
- `getGenImagesBySection()` helper maps generated images to correct sections
- Puppeteer renders each slide HTML to 1920×1080 PNG
- pdf-lib merges all PNGs into final PDF

`packages/graph-db/src/queries.ts`

Neo4j query library: CRUD operations, graph traversal, cultural queries (find similar artists, talent matching for campaigns), visualization data export.

`packages/graph-db/src/schema.ts`

13 unique constraints + fulltext index. Run to initialize a fresh Neo4j database.

`apps/api/src/index.ts`

Express server with all routes. Handles multipart uploads (multer), campaign state management, persistence to disk, graph visualization endpoint, ingestion endpoint.

`apps/web/src/app/create/page.tsx`

2-step campaign creation UI. Pinterest URL input with iframe preview, drag-and-drop image upload, artist photo upload, image curation grid with star-to-favorite.

`apps/web/src/app/campaign/[id]/page.tsx`

Campaign results page with sticky header, 4 tabs, download buttons. Touchpoints displayed inline in Overview tab as 2-column grid.

`apps/web/src/app/graph/page.tsx`

The Graft — 3D sphere visualization using Canvas 2D. ~750 lines. Fibonacci sphere distribution, perspective projection, camera controls, reference point input.

---

9. COSTS

Per Campaign

ComponentCost |----------|------| Claude Vision (115 images × $0.015)~$1.70 Claude Synthesis (16K tokens)~$0.15 Gemini Image Gen (25 × $0.02)~$0.50 **Total per campaign~$2.50-3.50**

Infrastructure

- Neo4j: Free (community edition, local)
- Hosting: TBD (not deployed yet, runs locally)

10. CURRENT STATE & KNOWN ISSUES

What Works

- ✅ Full campaign flow: upload → analyze → synthesize → generate images → build deck → download PDF/PPTX
- ✅ Pinterest scraping and image curation
- ✅ 25 AI-generated images per campaign across 6 sections
- ✅ Neo4j with ~116+ nodes and relationships
- ✅ Entity ingestion with data enrichment
- ✅ Campaign persistence to disk
- ✅ Graft 3D visualization with zoom, rotation, click-to-focus
- ✅ Successful test campaign: 125 images → 115/125 analyzed → 45-slide deck at 21.5MB

Known Issues

- **Dev server port conflicts**: ts-node-dev hot reload constantly fights with EADDRINUSE on port 4000. Need to kill processes and restart frequently.
- **Possible duplicate Tyler node**: Seed "tyler-the-creator" vs ingested "Tyler, The Creator" — needs deduplication in Neo4j.
- **Mass ingestion may be incomplete**: 114 entities from mass-ingest.txt were being ingested via background process. Check Neo4j node count to verify.
- **API keys exposed in chat**: OpenAI, Anthropic, and Gemini keys were shared in Telegram chat. ALL NEED TO BE ROTATED.
- **No auth/billing**: Prototype only — no user accounts, no payment.
- **No History page**: The nav links to "/" for History but the landing page no longer shows campaigns. Need a dedicated `/history` route.
- **No error recovery in campaign flow**: If analysis fails mid-way, the campaign gets stuck.
- **Batch analyzer JSON truncation**: Sometimes Claude's synthesis response gets truncated. There's a JSON repair fallback but it's not bulletproof.

API Keys That Need Rotation

- **Anthropic**: Was shared in Telegram chat
- **Gemini**: `AIzaSyCL0WLN1pVKkiN2WkwZMYdpNK7T_9jeO-w` — was shared in chat
- **OpenAI**: `sk-proj-qGu2...` — was shared in chat (not currently used in the project but should be rotated)

---

11. WHAT'S NEXT (PRIORITIZED)

Immediate (Polish for Demo)

- Build `/history` page showing past campaigns
- Fix dev server stability (consider switching from ts-node-dev to tsx watch)
- Deduplicate Tyler node in Neo4j
- Verify mass ingestion completed, check total node count
- Test full campaign flow end-to-end with the expanded 25-image generation
- Rotate all exposed API keys

Short-term (Product)

- Campaign editing — re-run with different inputs or revisions
- Interactive graph in campaign context — see why the AI made specific connections
- Talent recommendation cards in deck (photographer X would be great for this based on graph data)
- Loading/generation experience — images appearing cinematically, not a progress bar
- Drag-and-drop mood board canvas (Polaroids-on-a-table feel)

Medium-term (Growth)

- User accounts and authentication
- Billing (Stripe)
- Deploy (Vercel for frontend, Railway/Fly for API, Neo4j Aura for graph)
- More data sources: Chartmetric ($300/mo), additional social APIs
- Figma/Canva plugin integration
- Google Slides direct export (in addition to PPTX)

Long-term (Vision)

- Real-time collaboration on campaigns
- Graph that learns from campaigns (every campaign enriches the graph)
- API for third-party integrations
- White-label for agencies
- Mobile app for quick mood board capture

---

12. DESIGN DECISIONS LOG

DecisionReasoning |----------|----------| YouTube over Spotify for audio analysisSpotify audio features API deprecated late 2024 No auth for prototypeCore engine only — prove the product works first Keep Neo4j despite being overkill for MVPGraph visualization is core to the pitch Programmatic HTML/CSS templates over static files29 templates as TypeScript functions — more flexible, data-driven pinterest-dl (Python CLI) for scrapingNo official API needed, subprocess call Gemini for image generationFast, cheap (~$0.02/image), supports multimodal input (style + identity refs) User curates deck references manuallyAI auto-selection was considered but manual gives user control Dark themeAndrew's preference after trying light theme Campaign persistence to JSON filesSimple, no additional database needed PPTX export over PDF-onlyOpens in Keynote + Google Slides + PowerPoint — one format for all Monochrome GraftClean, doesn't distract from the data Enrichment as non-blocking stepIf external APIs fail, ingestion still succeeds via Claude data Ingestion skips existing entities by defaultPrevents duplicates during batch runs 25 AI images organized by deck sectionEach section (photography, video, merch, etc.) gets its own generated visuals

---

13. RUNNING THE PROJECT

Prerequisites

- Node.js 22+
- Neo4j (Homebrew: `brew install neo4j`, `brew services start neo4j`)
- Python 3 with pinterest-dl (`pip install pinterest-dl`)
- Puppeteer (installed via npm, needs Chrome/Chromium)

Setup

```
cd /Users/moshii/.openclaw/workspace/cultural-graph
cp .env.example .env

FILL IN ANTHROPIC_API_KEY AND GEMINI_A

npm install
npx turbo dev
```

Initialize Neo4j

```
SET PASSWORD TO "CULTURALGRAPH" VIA NE

THEN RUN SCHEMA + SEED:

npx tsx scripts/import-seed.ts
```

Dev Servers

```
npx turbo dev

FRONTEND: HTTP://LOCALHOST:3000

API: HTTP://LOCALHOST:4000
```

Ingest Entities

```
npx tsx scripts/ingest.ts "Artist Name"
npx tsx scripts/ingest.ts --file data/seed/mass-ingest.txt
```

This document was generated on February 24, 2026. It represents the complete state of the Culture & Context project after ~2 days of intensive development.